NODEBASE - INTERVIEW PREPARATION

Technology Choices

Why each technology was selected, what alternatives existed, and what trade-offs the stack carries.

12 questions, answered in simple language

Each answer is written to be spoken in an interview. It is based on the actual codebase: Next.js 15, React 19, TypeScript, tRPC, Prisma, PostgreSQL, Better Auth, Polar, Inngest, BullMQ, Redis and React Flow.

Prepared in 2026

1. Why did you choose Next.js 15?

I chose Next.js because Nodebase needs both a rich client application and a server that can handle authentication, webhooks, tRPC calls and page rendering. Next.js gives me both in one codebase. The App Router lets me use server components for data loading and client components only where interactivity is needed, which keeps the editor fast and the initial pages simple.

The file-based routing also maps naturally to the product. I can separate the authentication routes from the dashboard routes with route groups, and I can place API routes for Better Auth, tRPC, Inngest and webhooks in the same project. That avoids running a separate Express server and then dealing with CORS, shared types and two deployment pipelines.

Server-side prefetching was another important reason. Pages can call `requireAuth`, prefetch data through tRPC, and hydrate the client cache before rendering. The editor then reads the workflow with a suspense query and paints immediately instead of showing a loading spinner and then making a second request.

The trade-off is that Next.js adds complexity. I have to understand server versus client boundaries, caching, and the limitations of serverless execution time. That is why long-running workflow execution is handled by Inngest and background jobs by BullMQ rather than by a normal request.

2. Why did you choose TypeScript over JavaScript?

TypeScript was important because Nodebase handles a lot of dynamic data. Node configuration is stored as JSON, workflow context is an arbitrary object, and tRPC responses are inferred from server procedures. Without types, it would be very easy to read a field that does not exist, pass the wrong shape to an executor, or break the client when a server response changes.

TypeScript gives me end-to-end confidence. Prisma generates types from the database schema, Zod validates runtime input, tRPC infers request and response types, and React components receive typed props. If I rename a field or change a procedure input, the compiler points to the places that need updating. That is especially valuable in a project with many node types, because each node has its own data shape.

It also improves the developer experience. Autocomplete shows the available fields on a Prisma result or a tRPC response, and editor tooling catches mistakes before I run the app. The cost is a build step, occasional complex generic types, and some friction when dealing with third-party libraries. I think that cost is small compared with the number of runtime bugs the compiler prevents.

For a project of this size, plain JavaScript would have worked for the first version, but it would have made refactoring and adding node types much riskier. TypeScript is now part of how I reason about the system.

3. Why did you choose Prisma ORM?

I chose Prisma because the domain is relational and the schema is central to the product. Workflows own nodes, nodes own connections, executions belong to workflows, and credentials belong to users. Prisma lets me describe those relationships in one schema file and generates a typed client from it. That keeps the database model readable and makes the TypeScript code match the actual tables.

Prisma also gave me migrations and a reliable way to apply schema changes. I could add a node type, change a column or add an index without hand-writing all the SQL. The generated client supports

relations, nested reads, cascading deletes and transactions, which I use heavily when saving a workflow. The `$transaction` API was especially useful because saving a graph must be all-or-nothing.

The client is also safer to use than string-based SQL. I get autocomplete for fields and relations, and common mistakes become compile errors instead of runtime failures. For example, querying executions through the workflow relation is type-checked, which matters because ownership is enforced through that relation.

Prisma is not perfect. It adds a generated client, can increase cold-start cost, and sometimes hides the SQL that is actually executed. For the row-level lock in workflow creation I used `$queryRaw`, and for complex analysis I would not hesitate to use raw SQL. But for the core CRUD and transactional work, Prisma improved both speed and safety.

4. Why did you choose Neon PostgreSQL?

Neon was a practical choice because it provides managed PostgreSQL without requiring me to run and maintain a database server. PostgreSQL itself was the important decision; Neon was the hosting layer that made it easy to use during development and deployment. It offers a standard Postgres connection string, so the application does not depend on a proprietary database API.

Neon also supports branching, which is useful for development and testing. I can create a branch of the database for an experiment or a migration without affecting the main data. That is a feature I would use more heavily if the project had a larger team and a formal staging environment.

The serverless model and connection pooling were another advantage. Next.js can run in environments where many short-lived connections are created, and a normal Postgres server can struggle with that. Neon provides pooled connections that are better suited to serverless workloads. It also integrates well with Vercel-style deployments.

The trade-offs are cold starts on inactive databases, plan limits, and some dependence on the provider. For a production system I would add connection pooling through PgBouncer or the Neon pooler, monitor connection counts, configure backups and point-in-time recovery, and have a plan for moving to a larger managed Postgres instance if the workload grows.

5. Why did you use Inngest?

Workflow execution cannot happen inside a normal HTTP request. A run may take tens of seconds, call several external APIs, and need retries when one of them fails. I used Inngest because it provides durable step functions, so I do not have to build my own queue, state machine and retry logic for the execution engine.

Every long operation in the Inngest function is wrapped in `step.run`. Inngest records the result of each completed step, and if the function is retried, those steps return their memoized results instead of running again. That is what makes retries safe. The execution row is not inserted twice, and a node that already succeeded is not repeated just because a later node failed.

Inngest also gives me an event-driven interface. The rest of the application only needs to send a `workflows/execute.workflow` event. It does not need to know how the run is scheduled or retried. The `onFailure` hook lets me mark the execution FAILED even when the function body dies unexpectedly, and the realtime channels let me stream node status to the browser.

The trade-off is an external dependency and a development server. If Inngest is unavailable, new runs cannot be dispatched or executed. I accept that because building equivalent durability myself would be much more error-prone. Alternatives such as Temporal or Trigger.dev would solve a similar problem, but Inngest fit the event and step model very naturally.

6. Why PostgreSQL instead of MongoDB?

The core domain of Nodebase is relational. A workflow owns nodes, nodes are connected by edges, executions belong to workflows, and credentials belong to users. Those relationships need foreign keys, joins and cascading deletes. PostgreSQL enforces all of that in the database, so an edge cannot point to a node that does not exist, and deleting a workflow removes its graph and history automatically.

Saving a workflow also needs a real transaction. The graph is written across the Node and Connection tables, and a half-saved graph would be a broken product. PostgreSQL gives me multi-row ACID transactions, so the whole save can succeed or roll back together. MongoDB has improved multi-document transactions, but they are more limited and not the natural model of the database.

The execution table also benefits from relational constraints. `inngestEventId` is unique, which ties one execution row to one Inngest run and makes retries safe. Status is an enum. Ownership is reached through the workflow relation. Those guarantees are simple in PostgreSQL and would have to be enforced manually in MongoDB.

MongoDB is a good choice when documents are the unit of work and the schema changes constantly. In Nodebase, the flexible part is the per-node JSON configuration, and PostgreSQL already handles that with JSONB. So I get document-like flexibility for node data while keeping strict integrity for the graph and execution lineage. That hybrid model is exactly what the product needs.

7. Why Prisma instead of writing raw SQL?

Raw SQL gives complete control, but it also means writing the same joins, inserts and relation handling repeatedly by hand. Prisma gives me a typed client generated from the schema, so most database operations are short, readable and checked by TypeScript. That matters when many procedures touch the same tables and when the schema changes during development.

Prisma also handles relations and cascades in a way that matches the domain. I can include a workflow's nodes and connections in one query, or create a credential and rely on the foreign key to the user. Transactions are expressed as a callback, which makes the save logic easy to read. Migrations are generated from schema changes, so the database and the code stay in sync.

Raw SQL is still useful, and I do use it where Prisma is not the best tool. The workflow creation limit uses `$queryRaw` to run `SELECT ... FOR UPDATE`, because I need a row-level lock to prevent two concurrent creates from both passing the free-tier check. For complex reporting or database-specific features I would also use raw SQL.

The trade-off is that Prisma hides some SQL and adds a generated client to the build. There is a learning curve for advanced queries and performance tuning. But for a product where correctness and developer speed matter more than squeezing every millisecond from a query, Prisma is the better default. I treat raw SQL as an escape hatch, not as the main interface.

8. Why Next.js instead of React with Express?

React with Express is a perfectly valid architecture, but it would have meant maintaining two applications and two deployment processes. With Next.js, the React UI, the tRPC API, the authentication routes, the webhook routes and the Inngest serve route all live in one project. That reduces setup work and makes it easier to share types and configuration.

Next.js also gives me server rendering and server components. Pages can authenticate the user, prefetch data and render the first version on the server. That is better for perceived performance and avoids a client-side waterfall where the page loads, then fetches, then renders. The editor is still a client component because it needs React Flow, but the surrounding pages benefit from the server model.

Using one framework also avoids CORS and cross-service authentication complexity. The browser calls tRPC on the same origin. Better Auth sets cookies on the same origin. Webhooks and the Inngest route are part of the same deployment. In an Express setup I would have to configure those boundaries explicitly.

The trade-off is coupling to Next.js conventions. Some developers prefer the freedom of a separate API server, and very long-running work does not belong in a Next.js request. I handled that by moving execution to Inngest and email and cron work to BullMQ workers. So Next.js handles interactive traffic, while the heavy asynchronous work runs in separate processes.

9. What alternatives did you consider for each technology?

For the frontend and server framework, I considered Remix, SvelteKit and a separate React frontend with Express. Next.js won because of its maturity, App Router, server components and the large ecosystem around authentication and deployment. For the language, the alternative was plain JavaScript, but TypeScript was clearly better for a graph-based product with many data shapes.

For the database layer, I considered Drizzle, Kysely, TypeORM and raw SQL. Drizzle is lighter and closer to SQL, and it is a strong alternative. I chose Prisma because its schema, migrations and relation handling were mature and the typed client made the transaction code easy to write. For the database itself, I considered MongoDB, MySQL, Supabase and a managed RDS instance. PostgreSQL on Neon won because the domain is relational and Neon made hosting simple.

For execution and background jobs, I considered Temporal, Trigger.dev, a custom BullMQ worker, and direct API calls. I chose Inngest because durable steps, retries, failure hooks and realtime channels were built in. BullMQ is still used for the simpler email and cron workloads, where a full workflow engine would be unnecessary.

For auth I considered Auth.js, Clerk and Supabase Auth. I chose Better Auth because it supports email and password, OTP, Google, GitHub, Prisma and Polar in one place. For billing I considered Stripe directly, but Polar's subscription state and Better Auth plugin reduced the amount of billing code I had to write. For the canvas I considered other graph libraries, but React Flow had the best balance of custom nodes and documentation.

10. What are the disadvantages of your technology stack?

The stack is powerful, but it has real trade-offs. Next.js introduces server and client boundaries, caching rules and deployment-specific constraints that can be confusing. Prisma adds a generated client and can hide inefficient SQL. Inngest, Neon, Upstash, Polar and the SMTP provider are all external services, so the system depends on several vendors being available and correctly configured.

There is also an operational surface that a smaller stack would not have. I need to run the Next server, the Inngest dev server, the email worker and the cron worker locally. In production, I need to monitor queue depth, failed jobs, database connections, retries and SMTP delivery. Upstash Redis uses an eviction policy that is not ideal for queue durability, which is a risk for jobs that must not be lost.

The current codebase also has some internal duplication. Each node type has its own realtime channel file and token action, and those files are almost identical. The execution engine passes one mutable context object, which is simple but not strongly typed. The editor's save path has a special rule that ignores empty node data to avoid wiping configuration, which also makes it impossible to clear a node through the normal form.

Finally, the project does not yet have automated tests or a mature CI/CD pipeline. That is a gap in the stack usage rather than a flaw in the technologies themselves, but it matters for production readiness. I would address it before scaling the product to real customers.

11. How do the different technologies communicate with each other?

The browser communicates with the Next.js server over HTTPS. Interactive data operations use tRPC on the `/api/trpc` route, authentication uses Better Auth on `/api/auth/[...all]`, and external systems call the webhook routes. Pages are rendered by Next.js server components, which call tRPC helpers directly and hydrate the results into React Query on the client.

Inside the server, tRPC procedures call Prisma, and Prisma talks to PostgreSQL over the database connection. Better Auth also uses Prisma through its adapter, so users, accounts, sessions and verification rows live in the same database. Polar is called through its SDK to check subscription state. Credentials are decrypted with Cryptr only inside executors, using the encryption key from the environment.

Workflow execution is event-driven. The application sends an event to Inngest, and Inngest calls the registered function on the `/api/inngest` route. The function uses Prisma to read the graph and write execution rows, and it uses the AI SDK, `ky`, the Telegram API, Discord, Slack and the Google Sheets API to perform node work. Executors publish node status through Inngest realtime channels.

Background jobs use BullMQ over an ioredis-compatible connection to Upstash Redis. The email worker sends OTP messages through SMTP. The cron worker receives scheduled ticks and sends workflow events back to Inngest. Upstash Redis also stores rate-limit counters and idempotency keys. The browser subscribes to realtime status using a short-lived token obtained from a server action, which avoids exposing a long-lived secret to the client.

12. Which parts of your application could be replaced without changing the entire architecture?

The UI layer is replaceable because the server exposes tRPC procedures rather than page-specific endpoints. A different frontend, such as a mobile app or another React framework, could call the same procedures. The editor itself could be replaced with a different canvas library as long as it produces the same node and edge shape that `workflows.update` expects.

The authentication provider is also replaceable. The rest of the application mostly depends on the session containing a user id. If I moved from Better Auth to another provider, I would need to change the auth configuration and the procedure middleware, but the workflow, credential and execution routers would stay largely the same. The same is true for billing: Polar is isolated behind the premium procedure and the subscription helper.

The execution backend could be replaced too. The domain depends on the event name, the executor contract and the execution table, not on Inngest specifically. A Temporal, Trigger.dev or custom BullMQ-based runner could implement the same interface. The node executors would not need to change if the new runner provided step tools and a publish function.

At the infrastructure level, Neon could be replaced by any PostgreSQL provider, Upstash could be replaced by another Redis instance, and the SMTP provider could be replaced by another email service. The stable contracts are the database schema, the tRPC router surface, the executor interface and the workflow event. Keeping those contracts clean is what makes the rest replaceable.